



WeatherForecast Service

Многопользовательский сервис прогноза погоды с системой email-подтверждения и ролевой моделью доступа. Современное RESTful API для управления персонализированными прогнозами погоды.

Технологический стек: Spring Boot • JWT Authentication • MySQL • Java Mail • Swagger/OpenAPI

GitHub: github.com/YuriyDolgikh/weatherforecast

Системные требования: Java 17+, Maven 3.6+, MySQL 8.0+, API ключ [Weatherbit.io](https://weatherbit.io), SMTP сервер

Бизнес-задачи и функциональные требования

Проект решает задачу предоставления персонализированного сервиса прогноза погоды с расширенными возможностями управления данными и безопасностью.



Прогноз на 7 дней

Детальные прогнозы погоды для выбранных городов с почасовой детализацией и основными метеопараметрами



Избранные города

Персональная система управления списком городов для каждого пользователя с быстрым доступом



Автообновление

Scheduled tasks для фонового обновления данных каждые 4 часа через Weatherbit.io API



Email-верификация

Двухэтапная регистрация с отправкой кода подтверждения на электронную почту пользователя

Безопасность

- JWT-аутентификация с токенами доступа
- Ролевая модель: USER и ADMIN
- Защищенные endpoints с авторизацией

Качество кода

- Полное тестовое покрытие (Unit + Integration)
- Swagger документация API
- Глобальная обработка исключений



Технологический стек проекта



Backend Framework

Java 17+ с использованием современных возможностей языка

Spring Boot 3.x для быстрой разработки и конфигурации

Spring Data JPA для работы с персистентным слоем и MySQL



Security & Auth

Spring Security для комплексной защиты приложения

JWT токены для stateless аутентификации

Ролевая модель с разграничением доступа



Messaging

Spring Mail для отправки email-уведомлений

FreeMarker шаблонизатор для HTML-писем

JavaMailSender для SMTP интеграции

Тестирование

- JUnit 5
- Mockito
- Spring Boot Test

Планировщик

- Spring Scheduler
- Cron выражения
- Async обработка

Документация

- Swagger UI
- OpenAPI 3.0
- REST спецификация

Регистрация и аутентификация пользователей

Двухэтапный процесс регистрации с email-подтверждением обеспечивает валидность учетных записей и защиту от ботов.



Регистрация

POST /api/public/register

Создание учетной записи с валидацией данных



Генерация кода

Создание уникального UUID для подтверждения



Отправка email

HTML-письмо с ссылкой подтверждения на FreeMarker шаблоне



Подтверждение

GET /api/public/confirmation

Переход в статус CONFIRMED

JWT аутентификация

После подтверждения регистрации пользователь получает доступ к защищенным endpoints через JWT токены. Токен выдается при успешной аутентификации и содержит информацию о роли пользователя.

Endpoint: POST /api/auth

Возвращает: Access Token с временем жизни

Роли: USER (доступ к личным данным), ADMIN (статистика и аналитика)



Пользовательские endpoints

API предоставляет полный набор операций для управления профилем, избранными городами и получения прогнозов погоды.

Управление профилем

- **GET /api/user** — получение данных профиля
- **PUT /api/user** — обновление информации
- **DELETE /api/user** — удаление учетной записи

Избранные города

- **GET /api/user/cities** — список моих городов
- **GET /api/user/cities/add? cityName={name}** — добавление города
- **GET /api/user/cities/delete? cityName={name}** — удаление города

Прогноз погоды

- **POST /api/user/forecast** — детальный прогноз на 7 дней
- **GET /api/user/forecast** — средняя погода на сегодня по избранным городам

📄 **Авторизация:** Все пользовательские endpoints требуют валидный JWT токен в заголовке Authorization. Каждый пользователь имеет доступ только к своим данным и городам.



Административные функции

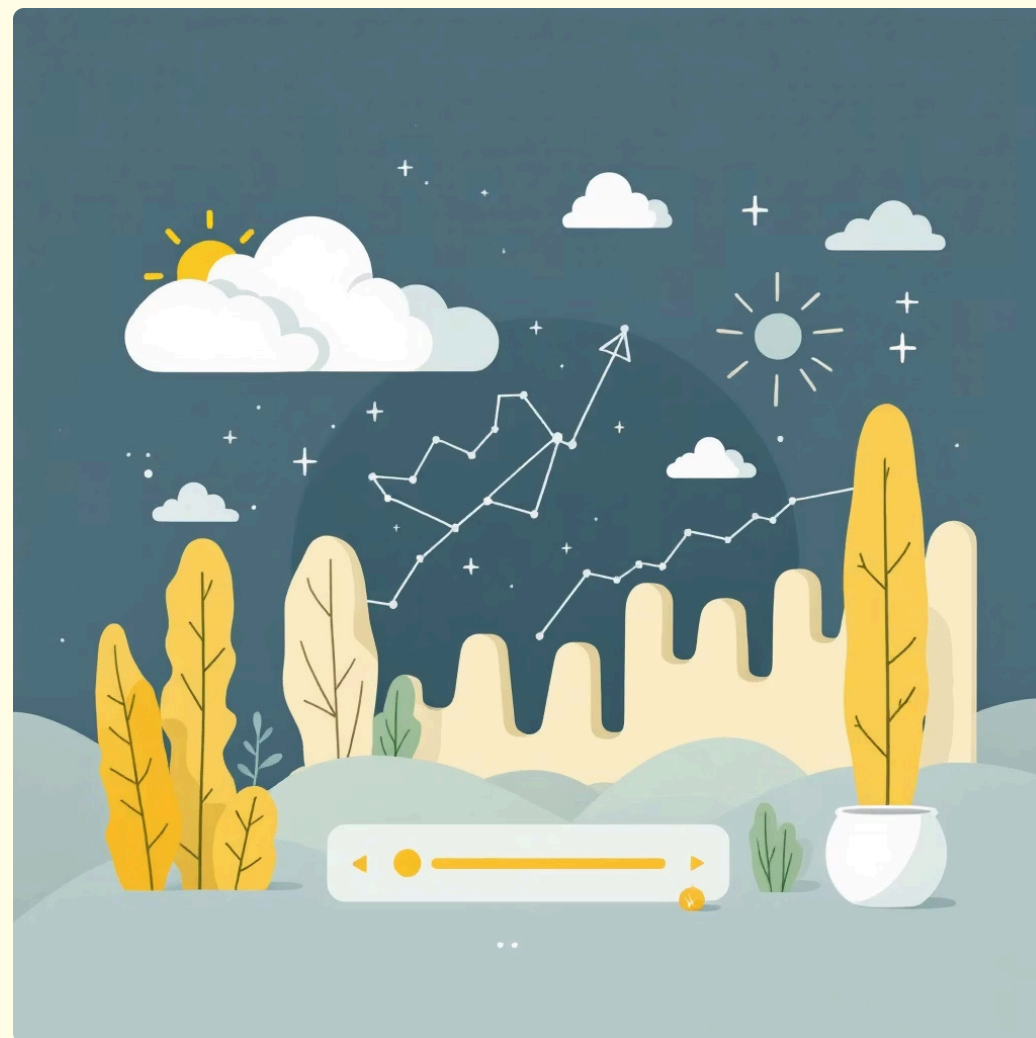
Расширенный набор endpoints для администраторов обеспечивает доступ к статистике, аналитике и управлению пользователями.

Статистика пользователей



- **GET /api/admin/findAll** – список всех зарегистрированных пользователей с детальной информацией
- **GET /api/admin/allFavoritesCities** – агрегированный список всех избранных городов в системе
- **GET /api/admin/allFavoritesCitiesByUser/{id}** – избранные города конкретного пользователя

Аналитика погоды



- **GET /api/admin/coldest** – определение самого холодного города среди всех избранных
- **GET /api/admin/warmest** – поиск самого теплого города в базе данных
- **GET /api/admin/maxPrecipitation** – город с максимальными прогнозируемыми осадками

Административные endpoints защищены дополнительным уровнем авторизации и доступны только пользователям с ролью ADMIN. Это обеспечивает безопасность чувствительных данных и аналитической информации.

Автоматическое обновление данных

Spring Scheduler

Фоновая задача автоматически обновляет прогнозы погоды для всех избранных городов пользователей каждые 4 часа.

```
@Scheduled(cron = "0 0 */4 * * *")
public void updateWeatherData() {
    // Получение избранных городов
    // Интеграция с Weatherbit.io
    // Сохранение с constraints
}
```



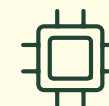
Актуальные данные

Пользователи всегда получают свежие прогнозы без необходимости ручного обновления. Система автоматически синхронизируется с Weatherbit.io API.



Кэширование в БД

Прогнозы сохраняются в MySQL с уникальными ограничениями по городу и дате, что исключает дублирование и оптимизирует запросы.



Фоновая обработка

Обновление происходит асинхронно в фоновом режиме, не влияя на производительность основного приложения и пользовательский опыт.

Система email-рассылок

Комплексная система подтверждения регистрации через email обеспечивает валидность учетных записей и защиту от автоматической регистрации.

1

Генерация кода

Создание уникального UUID для каждого пользователя при регистрации с ограниченным сроком действия

2

Создание письма

FreeMarker шаблонизатор формирует HTML-письмо с брендированным дизайном и ссылкой подтверждения

3

SMTP отправка

JavaMailSender отправляет письмо через настроенный SMTP сервер с обработкой ошибок

4

Подтверждение

Переход по ссылке `/api/public/confirmation?code={uuid}` активирует учетную запись

Технические детали

- Использование Spring Mail для интеграции
- HTML-шаблоны на FreeMarker
- Контроль времени жизни кодов
- Повторная отправка при необходимости

Безопасность

- Одноразовые коды подтверждения
- Временное ограничение действия
- Защита от перебора кодов
- HTTPS для защиты передачи данных

Стратегия тестирования

Комплексный подход к обеспечению качества кода включает несколько уровней тестирования для гарантии надежности и стабильности приложения.

Unit-тесты сервисов

Mockito для изоляции зависимостей и тестирования бизнес-логики каждого сервиса отдельно

- Тестирование Weather Service
- Проверка User Service логики
- Валидация Email Service

Интеграционные тесты

Spring Boot Test для проверки взаимодействия компонентов и REST контроллеров

- Тесты API endpoints
- Проверка работы с БД
- Интеграция с внешним API

Security тесты

Проверка правильности работы JWT аутентификации и авторизации для всех защищенных endpoints

- Валидация токенов
- Проверка ролевой модели
- Тестирование доступа

Покрытие кода

Высокий процент покрытия тестами гарантирует стабильность при рефакторинге и добавлении новых функций. Все критические бизнес-процессы покрыты автоматизированными тестами.

Граничные случаи

Особое внимание уделяется тестированию edge cases: невалидные данные, отсутствие соединения с API, некорректные токены, истекшие коды подтверждения.

Итоги и достижения проекта

WeatherForecast Service демонстрирует полный цикл разработки enterprise-уровня Spring Boot приложения с современными практиками и архитектурными решениями.

10+

REST endpoints

Для пользователей и администраторов

100%

Test Coverage

Unit и интеграционные тесты

2

Роли доступа

USER и ADMIN с разграничением

7

Дней прогноза

Детальные метеоданные



Полнофункциональное Spring Boot приложение

с современной архитектурой и best practices



Двухэтапная регистрация

с email-подтверждением через JavaMailSender и FreeMarker



JWT аутентификация

с ролевой моделью доступа и защищенными endpoints



Персонализированные прогнозы

система избранных городов для каждого пользователя



Автоматизация

Spring Scheduler для фонового обновления данных каждые 4 часа



Административная панель

с расширенной статистикой и аналитикой погодных данных

Технологическое превосходство: проект демонстрирует глубокое понимание Spring экосистемы, принципов безопасности, работы с внешними API и комплексного тестирования. Полная документация Swagger и обработка ошибок делают API production-ready.